Activity: k-Nearest Neighbors

Math 437

```
library(tidyverse)
library(tidymodels)
library(kknn) # for the actual model fitting
library(forcats) # tidyverse for factors

Restrain <- readr::read_csv(here::here("C:/Users/aocho/Downloads/Restrain.csv"))

Restrain2 <- Restrain |> mutate(SubCategory =
  fct_collapse(SubCategory,
    savory = c("Bakery (savoury)", "Savoury snacks"),
    sweet = c("Bakery (sweet)", "Biscuits", "Confectionery", "Desserts"),
    takeout = c("Takeaway (chain)", "Takeaway (generic)"),
    fruit_veggie = c("Fruits", "Vegetables")
  )
)

set.seed(1880)
Restrain_split <- initial_split(Restrain2, prop = 0.80)
# can include a strata = "variable_name" argument to do stratified random training/test sampl

Restrain_train2 <- training(Restrain_split)
Restrain_test2 <- testing(Restrain_split)
```

k-Nearest Neighbors Algorithm

1. Explain how a k-nearest neighbors classification algorithm works.

The model to run k-nearest neighbors is `nearest_neighbor`. Because k-nearest neighbors works with both regression and classification, we must specify the mode. We must also specify the number of neighbors to choose.

```
K <- 10 # 10-nearest neighbors
knn_model <- nearest_neighbor(
  mode = "classification",
  neighbors = K,
  dist_power = 2
)
```

2. Briefly explain how Minkowski distance is calculated and what the `dist_power` argument refers to.

```
knn_recipe <- recipe(
  SubCategory ~ Taste + Cravings + Healthiness,
  data = Restrain_train2
) |>
  step_normalize(all_numeric_predictors()) #|>
# step_dummy(all_nominal_predictors())
```

3. Why should we center and scale the numerical variables?
4. Why do categorical variables need to be converted to indicator variables for this algorithm?

```
knn_wflow <- workflow() |>
  add_model(knn_model) |>
  add_recipe(knn_recipe)

knn_fit <- knn_wflow |>
  fit(data = Restrain_train2)
knn_fit
```

```
== Workflow [trained] ==========================================================
Preprocessor: Recipe
Model: nearest_neighbor()

-- Preprocessor ----------------------------------------------------------------
1 Recipe Step

* step_normalize()

-- Model -----------------------------------------------------------------------

Call:
```

```
kknn::train.kknn(formula = ..y ~ ., data = data, ks = min_rows(10,     data, 5), distance = 

Type of response variable: nominal
Minimal misclassification: 0.2163462
Best kernel: optimal
Best k: 10
```

Notice that we are using the `train.kknn` function in the `kknn` package instead of the `knn` function from the `class` package (as done in the ISLR Chapter 4 lab).

Making Predictions

As usual, we use the `predict` function to make predictions:

```
knn_predictions <- knn_fit |>
  predict(new_data = Restrain_test2)
knn_predictions
```

```
# A tibble: 104 x 1
   .pred_class
   <fct>
 1 sweet
 2 sweet
 3 sweet
 4 sweet
 5 sweet
 6 sweet
 7 sweet
 8 sweet
 9 sweet
10 sweet
# i 94 more rows
```

```
knn_predictions_probs <- knn_fit |>
  predict(new_data = Restrain_test2, type = "prob")
knn_predictions_probs
```

```
# A tibble: 104 x 4
   .pred_savory .pred_sweet .pred_fruit_veggie .pred_takeout
          <dbl>       <dbl>              <dbl>         <dbl>
```

```
 1      0.195         0.543                 0        0.262
 2      0             0.966                 0        0.0341
 3      0.00843       0.992                 0        0
 4      0.321         0.553                 0        0.126
 5      0.0624        0.607                 0        0.331
 6      0.218         0.656                 0        0.126
 7      0             0.631                 0        0.369
 8      0             0.820                 0        0.180
 9      0.180         0.514                 0        0.306
10      0             0.758                 0        0.242
# i 94 more rows
```

1. We used 10-nearest neighbors! Why don't we get nice round percentages with these predictions?

We can also use `augment` to add the predictions to the original data frame:

```
knn_predictions_df <- knn_fit |>
  augment(new_data = Restrain_test2)
knn_predictions_df |>
  dplyr::select(
    Food,
    SubCategory,
    .pred_class,
    .pred_savory,
    .pred_sweet,
    .pred_takeout,
    everything()
  ) |>
  slice(51:54)
```

```
# A tibble: 4 x 11
  Food            SubCategory .pred_class .pred_savory .pred_sweet .pred_takeout
  <chr>           <fct>       <fct>              <dbl>       <dbl>         <dbl>
1 Chicken McNugg~ takeout     takeout            0.288     0.00843         0.704
2 Filet-o-Fish    takeout     sweet              0.368     0.389           0.243
3 Double Quarter~ takeout     takeout            0.277     0.174           0.549
4 Crunchie McFlu~ takeout     sweet              0         0.758           0.242
# i 5 more variables: .pred_fruit_veggie <dbl>, Category <chr>, Taste <dbl>,
#   Cravings <dbl>, Healthiness <dbl>
```

2. Explain why the "Filet-o-Fish" (row 52) is predicted to be sweet.

We can obtain the confusion matrix and see what categories get confused with each other:

```
knn_predictions_df |>
  conf_mat(
    truth = SubCategory,
    estimate = .pred_class
  )
```

```
             Truth
Prediction    savory sweet fruit_veggie takeout
  savory           0     1           0       2
  sweet            8    38           0       7
  fruit_veggie     0     0          18       0
  takeout          5     1           0      24
```

Tuning k-Nearest Neighbors

In k-nearest neighbors, we have two hyperparameters that we can tune: the number of neighbors and the distance metric.

Remember that we tune hyperparameters using cross-validation. Here we'll use repeated 5-fold cross-validation.

```
set.seed(145)
Restrain_kfold <- vfold_cv(Restrain_train2,
                      v = 5, repeats = 2,
                      strata = SubCategory)
Restrain_kfold
```

```
#  5-fold cross-validation repeated 2 times using stratification
# A tibble: 10 x 3
   splits           id      id2
   <list>           <chr>   <chr>
 1 <split [331/85]> Repeat1 Fold1
 2 <split [332/84]> Repeat1 Fold2
 3 <split [332/84]> Repeat1 Fold3
 4 <split [334/82]> Repeat1 Fold4
 5 <split [335/81]> Repeat1 Fold5
 6 <split [331/85]> Repeat2 Fold1
 7 <split [332/84]> Repeat2 Fold2
 8 <split [332/84]> Repeat2 Fold3
```

```
 9 <split [334/82]> Repeat2 Fold4
10 <split [335/81]> Repeat2 Fold5
```

Here we'll tune both the number of nearest neighbors and the distance power.

```
knn_model_tune <- nearest_neighbor(
  mode = "classification",
  neighbors = tune(),
  dist_power = tune()
)

knn_params <- knn_model_tune |>
  extract_parameter_set_dials()
knn_params
```

```
Collection of 2 parameters for tuning
```

```
 identifier       type    object
  neighbors  neighbors nparam[+]
 dist_power dist_power nparam[+]
```

Let's set up our search grid using `expand.grid`:

```
knn_param_grid <- expand.grid(
  neighbors = seq(2, 62, by = 10),
  dist_power = c(1, 2, 5)
)
knn_param_grid
```

```
   neighbors dist_power
1          2          1
2         12          1
3         22          1
4         32          1
5         42          1
6         52          1
```

```
7          62          1
8           2          2
9          12          2
10         22          2
11         32          2
12         42          2
13         52          2
14         62          2
15          2          5
16         12          5
17         22          5
18         32          5
19         42          5
20         52          5
21         62          5
```

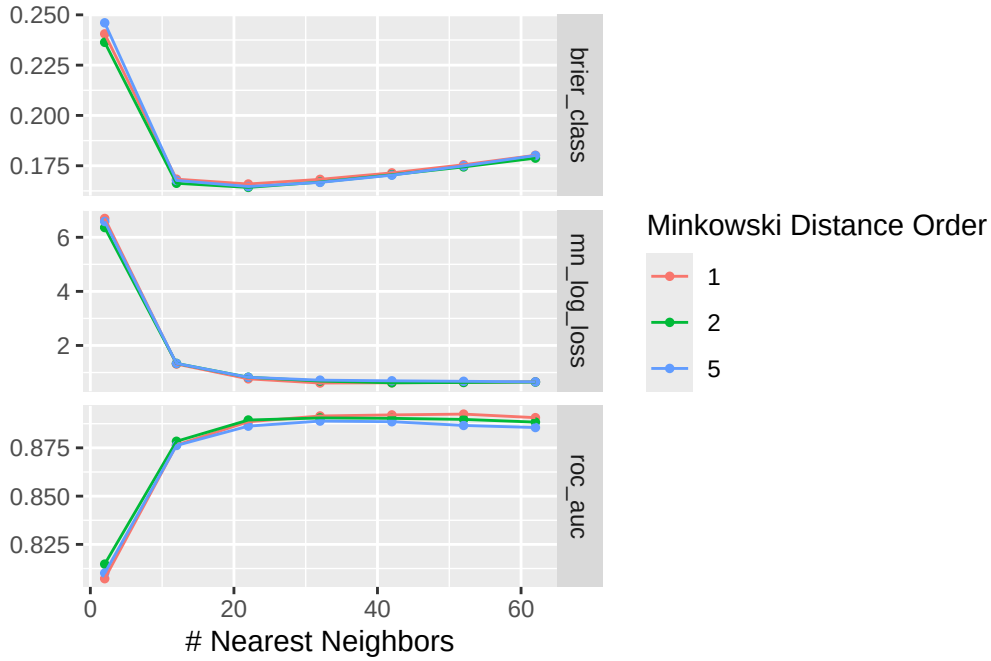
```
knn_wflow_tune <- workflow() |>
  add_model(knn_model_tune) |>
  add_recipe(knn_recipe)
```

Recall that we search over the entire grid using `tune_grid`:

```
knn_tune <- knn_wflow_tune |>
  tune_grid(
    resamples = Restrain_kfold,
    grid = knn_param_grid,
    metrics = metric_set(roc_auc, brier_class, mn_log_loss)
)
```

1. Explain the choice of `k` in terms of the bias-variance tradeoff.

```
autoplot(knn_tune)
```

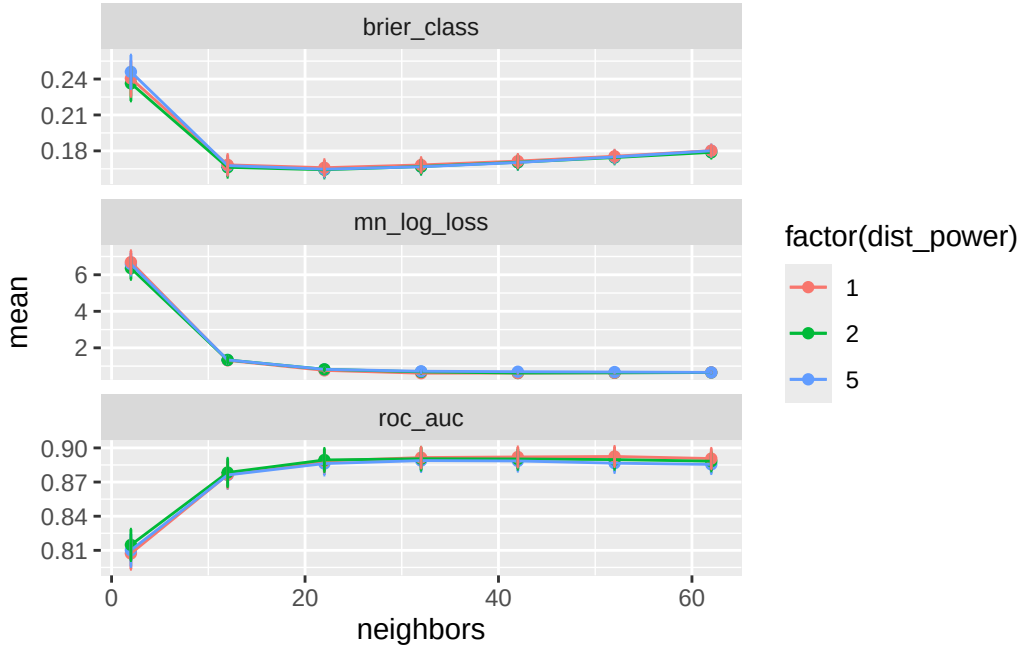


Since we get one tibble out for each fold, we can use `bind_rows` to make the output look a bit nicer and calculate the mean and SE.

```
knn_metric_avg <- knn_tune$.metrics |>
  bind_rows(
    .id = "validation_set"
  ) |>
  group_by(neighbors, dist_power, .metric) |>
  summarize(
    mean = mean(.estimate),
    se = sd(.estimate)/sqrt(length(knn_tune$.metrics)),
    .groups = "drop"
  ) |>
  group_by(.metric) |>
  arrange(mean, .by_group = TRUE)

ggplot(knn_metric_avg,
       mapping = aes(
         x = neighbors,
         y = mean,
         color = factor(dist_power)
       )) +
  geom_point() +
```

```
  geom_line() +
  geom_errorbar(
    aes(x = neighbors, ymin = mean - se, ymax = mean + se),
    width = 0.2
  ) +
  facet_wrap(
    vars(.metric),nrow = 3,
    scales = "free_y"
  )
```



2. How does the graph of `brier_class` illustrate the bias-variance tradeoff? What combination of `dist_power` and `neighbors` would you select based on this cross-validation?

```
knn_model_final <- nearest_neighbor(
  mode = "classification",
  neighbors = 22,
  dist_power = 2
)
knn_wflow_final <- workflow() |>
  add_model(knn_model_final) |>
  add_recipe(knn_recipe)
```

```r
knn_fit_tuned <- fit(knn_wflow_final,
                    data = Restrain_train2)

knn_predictions_df_tuned <- knn_fit_tuned |>
  augment(new_data = Restrain_test2)
conf_mat(
  knn_predictions_df_tuned,
  truth = SubCategory,
  estimate = .pred_class
)
```

```
              Truth
Prediction     savory sweet fruit_veggie takeout
  savory            0     0           0       3
  sweet             9    39           0       6
  fruit_veggie      0     0          18       1
  takeout           4     1           0      23
```

```r
knn_predictions_df_tuned |>
  filter(SubCategory == "takeout", .pred_class == "fruit_veggie")
```

```
# A tibble: 1 x 11
  .pred_class  .pred_savory .pred_sweet .pred_fruit_veggie .pred_takeout Food
  <fct>               <dbl>       <dbl>              <dbl>         <dbl> <chr>
1 fruit_veggie       0.0592           0              0.559         0.382 Vegeta~
# i 5 more variables: Category <chr>, SubCategory <fct>, Taste <dbl>,
#   Cravings <dbl>, Healthiness <dbl>
```